

Application Development 2 Project Report

Student Details

Name: Kriti Tiwari

Roll Number: 24F2005768

Email: 24f2005768@ds.study.iitm.ac.in

About Me: I am a student at IIT Madras BS in Data Science and Programming. I am currently at diploma level. I am interested in arts and reading. I wish to be able to make meaningful contributions to the field of data science.

Project Details

Project Title: Hospital Management System

Project Statement: To design and build a web-based application that allows Admins, Doctors, and Patients to interact with the system based on their roles.

Approach: The app uses Flask on the backend, Vue.js on the frontend and Celery for batch jobs. The app allows for hospitals, doctors and patients to easily manage appointments and treatments, and request reports.

AI/ LLM Declaration

No use of AI / LLM.

Technologies and Frameworks Used

1. **Flask:** Backend framework
2. **Flask-Security:** User authentication
3. **Flask-RESTful:**
4. **Flask-Caching:** Caching of frequently used endpoints
5. **Vue.js:** Frontend framework
6. **SQLite:** Lightweight local database for storing user data
7. **SQLAlchemy:** Object Relational Mapper for SQLite Database
8. **Redis:** Cache store and message broker
9. **Celery:** For handling asynchronous tasks
10. **Jinja2:** For creating mail templates
11. **Datetime:** Convert string to date type object and store into the database
12. **Dateutil:** Relative difference between two dates

13. **smtplib**: For sending emails
14. **Chart.js**: For analytical charts
15. **Bootstrap**: Styling framework for frontend
16. **Bootstrap 5 Icons**: Icons
17. **Mailpit**: Fake smtp server for testing emails

Database Schema/ ER Diagram

Tables:

1. **User**: stores user name, password and their blacklist status..
2. **Role**: stores roles of the app.
3. **UserRole**: stores the role of each user.
4. **Admin**: stores admin data.
5. **Patient**: stores patient name, contact number, gender, date of birth, height, weight, status, profile picture.
6. **Department**: stores department name, description and status.
7. **Doctor**: stores doctor name, contact number, email, date of birth, description, gender, status, profile picture.
8. **Shift**: stores shift-wise data for doctors.
9. **Appointment**: stores data about appointments date time, doctor ID, patient ID and slot ID
10. **Treatment**: stores diagnosis, tests, prescription and notes provided by the doctor for an appointment.
11. **Slots**: stores the 15 minute sessions for doctors and patients.
12. **ProfilePictures**: stores the name, role and image name of images used throughout the app.

Relationships

- **One-to-One relationships**: User-Admin, User-Doctor, User-Patient, Appointment-Treatment, Appointment-Slot
- **One-to-Many relationships**: Patient-Slot, Patient-Appointment, Department-Doctor, Doctor-Slots, Doctor-Appointment, Shift-Slot, ProfilePictures-Department, ProfilePictures-Doctor, ProfilePictures-Patients
- **Many-to-Many**: User-Role, Shift-Doctor



Architecture and Features

Architecture Overview:

- /Backend/app.py - main Flask application entry point
- /Backend/models.py - database models using SQLAlchemy
- /Backend/caching_config.py - cache initialization and management

- /Backend/extensions.py - SQLAlchemy and Flask-Security initialization
- /Backend/mark_past_appointments.py - function to automatically mark past appointments as completed
- /Backend/upcoming_week.py - function to add missing shifts to the hospital
- /Backend/reports - user requested reports
- /Backend/resources - Flask API endpoints
- /Backend/scripts - script to fill database with dummy data
- /Backend/tasks - Celery tasks
- /frontend/public - images used in the application
- /frontend/src/components - reusable Vue components
- /frontend/src/pages - main Vue pages
- /frontend/src/router - Vue Router initialization and definition of routes
- /frontend/src/stores - Pinia store definition
- /frontend/src/App.vue - Main Vue component
- /frontend/src/main.js - Vue app initialization
- /frontend/src/utlis - JavaScript function for sending requests to backend

Implemented Features

- Admin can perform the following functionalities:
 - Add, update and delete Patient profiles
 - Add, update and delete Doctor profiles
 - Add, update and delete Departments
 - Search for patients, doctors and departments by their name, contact numbers or IDs
 - View all appointments
 - Blacklist patients and doctors
- Doctor can perform the following functionalities
 - View all assigned appointments
 - Mark appointments as completed or cancelled
 - Enter diagnosis, tests, notes and prescriptions for each appointment.
 - View complete medical history of the patients
 - Update and provide their availability for the next 7 days
 - Search for other doctors, all departments and the patients assigned to the doctor
 - Request monthly reports
- Patients can perform the following functionalities:
 - Register, login and update their profiles
 - View and search doctors by specialization or name
 - View, book, reschedule, or cancel appointments.
 - View their own appointment and treatment history.
 - Request treatment history
 - Receive appointment reminders

Additional Features:

- Interactive charts for all users

Video Presentation

https://drive.google.com/drive/folders/1AriNjggFqnbxrvvAOrXhNMhAwWyTkOc_?usp=sharing